

Labo 00 — De Linux-basis die Docker als bekend veronderstelt

Theorie — de kernel, processen, gebruikers, bestanden, de shell en net genoeg netwerk. De volgende labo's gebruiken dit alles zonder het nog uit te leggen.

Doelstellingen

- De drie lagen van een Linux-systeem kennen: kernel, system calls, userland.
- Een proces kunnen beschrijven: PID, ouder, omgeving, signalen, exitcode.
- Een regel van `ls -l` kunnen lezen — eigenaar, groep, permissies — en weten wat `root` wél mag en jij niet.
- Vlot met de shell werken: omgevingsvariabelen, `PATH`, redirections, pipes.
- Weten in welk Docker-labo elk van deze begrippen terugkomt.

1. Waarom een Linux-labo in een Docker-opleiding

Omdat een container **niets anders is dan Linux**. In labo 01 lees je dat "een container een geïsoleerd proces is". Aan die zin heb je alleen iets als je precies weet wat een proces is: iets met een nummer, een ouder, een omgeving en een manier om te sterven. Hetzelfde geldt voor de rest. Zonder UID's begrijp je niet wat "rootless" betekent, zonder poorten niet waarom een poort "al bezet" is, zonder mounts niet wat een volume doet. Dit labo leert je precies die woordenschat — meer niet — en elke sectie vertelt erbij in welk labo het begrip terugkomt.

→ GESCHIEDENIS

Unix ontstond in 1969 bij Bell Labs en legde twee ideeën vast die nog altijd alles bepalen: *alles is een bestand*, en *kleine programma's die je combineert*. In 1991 schreef een Finse student, Linus Torvalds, een vrije Unix-compatibele kernel: Linux. Ubuntu (2004) is er een **distributie** van: de Linux-kernel plus een zorgvuldig samengestelde userland. En sinds 2019 levert Microsoft zijn eigen Linux-kernel mee in Windows. Dat is WSL 2 — de machine waarop je deze labo's doet.

2. De kernel en de userland

Een Linux-systeem bestaat uit twee lagen. Onderaan zit de **kernel**, het enige programma dat de hardware aanraakt. Hij maakt processen aan, verdeelt CPU-tijd en geheugen, leest schijven en verstuurt netwerkpakketten. Al de rest — `bash`, `ls`, `java`, jouw applicatie — draait erbovenop en heet de **userland**. Tussen beide ligt één enkele grens: de **system calls** (*syscalls*). Een programma leest nooit zelf een bestand. Het vraagt de kernel om `open` en daarna `read`, en de kernel beslist of dat mag.

Die grens verklaart twee dingen die je voortdurend zult tegenkomen. Eén: portabiliteit. Een Linux-binary draait op elke distributie, want hij gebruikt alleen system calls, en die zijn overal identiek. Twee: controle. Omdat alles via de kernel passeert, hoeft de kernel maar een beetje te liegen ("jij bent proces 1", "dit is jouw `/`") om een proces te isoleren. Precies dat doet een container, zoals je in labo 01 zult zien.

→ WINDOWS / WSL

Een Linux-programma kan alleen met een Linux-kernel praten; de Windows-kernel spreekt een andere, incompatibele taal. **WSL 2** (*Windows Subsystem for Linux*) lost dat op door een echte Linux-kernel te draaien in een kleine VM die Windows beheert. Jouw Ubuntu 24.04 leeft in die VM: `uname -r` toont er `...microsoft-standard-WSL2`, de handtekening van de kernel die Microsoft bouwt. De Windows-schijf vind je terug onder `/mnt/c`.

3. Processen

Een **proces** is een draaiend programma: code, geheugen en een identiteit. De kernel geeft het een uniek **PID** (*process ID*) en onthoudt wie de ouder is, het **PPID**. Elk proces wordt door een ander proces gemaakt. Typ je `ls`, dan kloont je shell zichzelf (`fork`) en vervangt de kloon zich door `ls` (`exec`). Bij het opstarten lanceert de kernel één eerste proces, **PID 1** (`systemd` op Ubuntu), de voorouder van alle andere. Sterft een ouder vóór zijn kind, dan adopteert PID 1 de wees. Hou dat nummer in je achterhoofd: in een container wordt *jouw applicatie* PID 1, en dat brengt onverwachte verantwoordelijkheden mee (labo 03).

→ LINUX

Een **daemon** is een dienstproces. Het wordt bij het opstarten gelanceerd door `systemd`, hangt aan geen enkele terminal en draait op de achtergrond tot iemand het nodig heeft: `sshd` wacht op SSH-verbindingen, `cron` op de volgende geplande taak. Bij afspraak eindigt de naam op een "d". Onthoud dit woord: Docker is gebouwd rond een daemon, `dockerd`, en Podman onderscheidt zich net doordat het er geen heeft. Dat debat vult labo 01.

Elk proces eindigt met een **exitcode**: `0` betekent succes, al de rest is een mislukking. De shell bewaart de code in `$?`. Enkele afgesproken waarden: `1` algemene fout, `2` verkeerd gebruik, `126` bestand niet uitvoerbaar, `127` commando niet gevonden, `128 + n` gedood door signaal *n*.

Een proces "sluit" je namelijk niet af — je stuurt het een **signaal**, een genummerde melding die de kernel aflevert. Drie signalen moet je kennen:

Signaal	Nummer	Betekenis	Kan het proces het negeren?
<code>SIGTERM</code>	15	"Sluit netjes af"	Ja — het krijgt de kans om op te slaan en op te ruimen
<code>SIGKILL</code>	9	Onmiddellijke dood, afgedwongen door de kernel	Nee — en er wordt niets opgeruimd
<code>SIGINT</code>	2	Onderbreking via het toetsenbord (<code>Ctrl+C</code>)	Ja

ONTHOUDEN

`kill` betekent niet echt "doden" maar "een signaal sturen", en standaard stuurt het het beleefde `SIGTERM`. Een proces dat door `SIGKILL` sterft, eindigt met code `137` (`128 + 9`). Dat getal blijf je tegenkomen zolang je met Docker werkt: het is de handtekening van een container die hardhandig werd gestopt — vaak omdat het geheugen op was.

De kernel publiceert de toestand van elk proces in `/proc`, een map die er eigenlijk geen is: `/proc/1234/` beschrijft proces 1234 — commandoregel, omgeving, limieten — ter plekke gegenereerd, zonder ook maar één byte schijfruimte. `ps` doet niets anders dan dat uitlezen.

4. Gebruikers, groepen, permissies

Elk proces draait *a/s* iemand: een **gebruiker**, herkend aan een nummer, de **UID**, plus een of meer **groepen** (GID). De kernel werkt uitsluitend met nummers; de namen (`kevin` , `postgres`) komen uit het bestand `/etc/passwd` . De eerste gebruiker op een Ubuntu-systeem krijgt UID **1000**. De gebruiker `root` , UID **0**, is speciaal: de kernel weigert hem niets. Met `sudo` voer je één commando uit *a/s* root, en het systeem logt wie erom vroeg.

Elk bestand heeft een eigenaar, een groep en negen permissiebits. Je leest ze af in `ls -l` :

```
-rw-r----- 1 root shadow 1234 ... /etc/shadow
  |  |  |      |
  |  |  |      | eigenaar root, groep shadow
  |  |  |      |
  |  |  |      |  |  |  |
  |  |  |      |  |  |  | anderen: niets
  |  |  |      |  |  |  |
  |  |  |      |  |  |  | groep shadow: lezen
  |  |  |      |  |  |  |
  |  |  |      |  |  |  | root: lezen + schrijven
```

`r` staat voor lezen, `w` voor schrijven, `x` voor uitvoeren (bij een map: binnengaan). `chmod` past de bits aan, `chown` de eigenaar. Eén detail waar iedereen ooit invliegt: een script moet **uitvoerbaar** zijn (`chmod +x`) voor je het met `./script.sh` kunt starten. Zo niet, dan antwoordt de shell `Permission denied` , exitcode 126.

→ BEVEILIGING

De gouden regel: werk nooit als root, en gebruik `sudo` alleen voor het ene commando dat het echt nodig heeft. Dat is de Linux-versie van het principe van minimale rechten, en meteen het kernargument voor **rootless** Podman: jouw containers draaien straks onder UID 1000, niet onder UID 0. Wordt een applicatie gekraakt, dan heeft de aanvaller alleen jouw rechten (labo 01).

VALKUIL

De kernel vergelijkt **nummers**, geen namen. Een bestand dat in een container door UID 1000 wordt aangemaakt, is overal eigendom van UID 1000 — ook al verschilt de getoonde naam van systeem tot systeem. Klinkt evident, maar in labo 06 wordt dit dé klassieke hoofdbreker bij volumes.

5. Bestanden, de boom, mounts

Onder Unix *is alles een bestand*: documenten, maar ook schijven (`/dev/sdc`), de toestand van de kernel (`/proc`), zelfs sockets. Stations zoals `C:` of `D:` bestaan niet. Er is één boom, met `/` als wortel, en elke schijf of elk bestandssysteem wordt erin **gemount** — vastgemaakt aan een map. `findmnt /` vertelt je welke schijf de wortel levert; op WSL hangt de Windows-schijf aan `/mnt/c` . Mounten, unmounten en bestandssystemen stapelen: zo werken Docker-images en -volumes onder de motorkap (labo's 02 en 06).

De standaardmappen die je moet herkennen: `/etc` (configuratie), `/home` (jouw bestanden), `/usr/bin` (programma's), `/var` (data die groeit: logs, databanken), `/tmp` (kladruimte), `/proc` en `/sys` (vensters op de kernel).

6. De shell: omgeving, PATH, pipes

De **shell** (`bash`) is een gewoon proces met als taak andere processen te starten. Drie van zijn mechanismen zijn onmisbare Docker-kennis.

Omgevingsvariabelen. Elk proces begint zijn leven met een woordenboek van sleutel=waarde-paren, geërfd van zijn ouder: `HOME`, `PATH`, `LANG`, enzovoort. Een gewone shellvariabele (`MSG=hallo`) blijft lokaal; pas na `export MSG` komt ze in de omgeving van de kindprocessen terecht. Zo worden containers geconfigureerd: in labo 08 leest je Spring Boot-applicatie haar databankwachtwoord uit een omgevingsvariabele, nooit uit een bestand dat in de image zit.

→ JAVA

Een JVM is een gewoon proces: `java -jar app.jar` heeft een PID, een UID en een omgeving. Spring Boot leest die omgeving bij het opstarten, dus met `SERVER_PORT=9090` verander je de poort zonder de JAR aan te raken. `System.getenv("HOME")` in Java leest datzelfde geërfde woordenboek.

Het `PATH`. Typ je `ls`, dan doorzoekt de shell de mappen uit de variabele `PATH`, in volgorde, op zoek naar een uitvoerbaar bestand met de naam `ls`. `which ls` toont wat hij vond; `command not found` (code 127) betekent dat de zoektocht niets opleverde. Daarom start je een script uit de huidige map als `./script.sh` — de huidige map staat bewust niet in het `PATH`.

Redirections en pipes. Een proces heeft drie stromen: invoer (0, *stdin*), uitvoer (1, *stdout*) en fouten (2, *stderr*). De shell kan ze aansluiten waar jij wilt: `> bestand` leidt de uitvoer om, `2>` de fouten, `2>&1` voegt beide samen, en `command1 | command2` stuurt de uitvoer van het ene commando naar de invoer van het andere. Zulke pijplijnen bouw je in elk labo (`podman ps | grep ...`), en de logs van een container zijn gewoon zijn stromen 1 en 2, opgevangen (labo 03).

7. Net genoeg netwerk

Met drie ideeën haal je labo 07. **De interface:** de netwerkaansluiting van een machine, met een IP-adres. `lo`, de *loopback*-interface, draagt `127.0.0.1`, beter bekend als `localhost` — de machine die met zichzelf praat. **De poort:** een nummer van 1 tot 65535 dat de diensten op eenzelfde adres uit elkaar houdt. Op een gegeven poort kan maar één proces luisteren, en `ss -tlnp` toont wie waar luistert. **De privilegeregel:** poorten onder 1024 zijn voorbehouden aan root. Daarom luistert je testserver straks op 8080 en niet op 80, en daarom weigert rootless Podman `-p 80:80` (labo 07).

→ NETWERK

`curl` is hier het Zwitserse zakmes: het stuurt een HTTP-verzoek en toont het rauwe antwoord. `curl -i http://localhost:8080/` geeft je de statuscode (`200 OK`, `404` ...), de headers en de body. Dé manier om een API in een container te testen zonder browser.

→ WINDOWS / WSL

WSL 2 stuurt `localhost` automatisch door: een server die op poort 8080 luistert in Ubuntu, bereik je vanuit een **Windows**-browser op `http://localhost:8080`. Handig — maar vergeet niet dat deze doorschakeling een dienst van WSL is, niet iets wat Linux zelf doet.

8. In het bedrijf

Het hele container-ecosysteem is de industriële versie van deze begrippen. Een Spring Boot-server in productie komt neer op: een `java`-proces (PID), gestart door een applicatiegebruiker zonder extra rechten (UID), geconfigureerd via omgevingsvariabelen, met logs naar *stdout*, luisterend op poort 8080, en bij elke deployment gestopt met `SIGTERM`. De beheerder die een incident onderzoekt, gebruikt `ps`, `ss` en `curl`, kijkt naar `$?` en doorzoekt de logs met `grep`. Docker vervangt daar niets van — het verpakt het.

PODMAN

Podman trekt die redenering helemaal door: geen daemon, alleen *jouw* gebruiker (UID 1000) die processen start. Alles uit dit labo — UID's, signalen, `/proc`, niet-geprivilegieerde poorten — beschrijft exact wat Podman zonder `sudo` mag doen. Docker daarentegen steunt op een daemon die als root draait (`dockerd`). Dat verschil is het onderwerp van labo 01.

Onthouden

- De **kernel** controleert alles; programma's kunnen alleen **system calls** doen. Een proces isoleren betekent de kernel tegen dat proces laten liegen — het basisidee achter containers.
- Een **proces** heeft een PID, een ouder en een geërfde omgeving, en eindigt met een **exitcode**: `0` = succes, `137` = gedood door SIGKILL.
- `SIGTERM` vraagt; `SIGKILL` dwingt af. Een goed opgevoede dienst stopt op SIGTERM.
- De kernel denkt in numerieke **UID/GID**; `root` = UID 0 = onbeperkte rechten; `sudo` geeft ze één commando tegelijk.
- Er is één bestandsboom; schijven en virtuele bestandssystemen worden erin **gemount**; `/proc` is jouw venster op de kernel.
- **Omgevingsvariabelen** stromen van ouder naar kind; het `PATH` bepaalt welke commando's bestaan.
- Een dienst = een adres + een **poort**; `localhost` = deze machine; poorten onder 1024 zijn van root.

Woordenschat

kernel: het programma dat hardware en processen controleert. — **userland**: alles wat boven op de kernel draait. — **system call**: verzoek van een programma aan de kernel (`open` , `fork` ...). — **proces**: een draaiend programma, herkend aan zijn **PID**. — **PID 1**: het eerste proces, voorouder en voorged van alle andere. — **daemon**: dienstproces op de achtergrond, zonder terminal, beheerd door `systemd` (`sshd` , `dockerd`). — **signaal**: melding die naar een proces wordt gestuurd (`SIGTERM` , `SIGKILL`). — **exitcode**: het getal dat een proces bij zijn dood teruggeeft; `0` = succes; te vinden in `$?`. — **UID / GID**: gebruikers- en groepsnummers, de enige identiteiten die de kernel kent. — **root**: UID 0, vrijgesteld van elke permissiecontrole. — **mount**: een bestandssysteem vastmaken aan een map in de boom. — **/proc**: virtuele boom die de toestand van de kernel en van elk proces toont. — **omgevingsvariabele**: sleutel=waarde-paar dat kindprocessen erven. — **PATH**: de lijst mappen waarin de shell commando's zoekt. — **stdin / stdout / stderr**: de drie standaardstromen (0, 1, 2). — **pipe**: de uitvoer van het ene proces verbinden met de invoer van het andere. — **poort**: nummer dat een dienst op een IP-adres aanduidt. — **localhost**: `127.0.0.1`, het adres van de machine zelf.